

Retrieval-Augmented Generation for Legal and Regulatory Document Q&A: An Architecture Note

Why RAG, and not fine-tuning or a bare LLM wrapper

For a system that answers questions against a living body of legal or regulatory text — bylaws, policies, contracts — the right architecture is retrieval-augmented generation (RAG), not a fine-tuned model and not a general-purpose chatbot with no retrieval layer. The reasoning is structural rather than a matter of taste.

Three properties of the problem force the choice. First, the underlying documents are not static: bylaws are amended, policies are revised, and any system whose knowledge is frozen at training time will silently go stale. Second, correctness matters more than fluency: the system must answer *only* from what is actually written, not produce plausible-sounding legal prose that happens to be unsupported. Third, every answer needs to be auditable — a user (or a compliance reviewer) must be able to trace a claim back to the exact clause that produced it.

Fine-tuning fails all three. It bakes a snapshot of the documents into the model’s weights, so an amendment the next day requires retraining rather than a document re-index. It also blurs the line between “what the source says” and “what the model has learned to generate,” which is precisely the failure mode one is trying to avoid in a legal context — the model can produce fluent text that is *not* grounded in any specific passage, and there is no mechanism for citation because the source text no longer exists as a separate, addressable object once it has been absorbed into weights.

A plain chatbot on top of a general LLM fails for the same underlying reason: with no retrieval step, the model is reasoning from its pretraining distribution, not from the target documents. It will produce text that reads as authoritative regardless of whether it is actually correct for the specific bylaws in question.

RAG works because it separates two distinct problems that a fine-tuned model conflates: *finding the right passage* (a search/retrieval problem) and *explaining it in plain language* (a generation problem). Keeping these separate is what makes the system re-indexable when a document changes, restrictable to only the supplied text, and traceable to a specific source at answer time.

The pipeline

The system has four stages.

1. Ingestion. Each document is split into chunks — a paragraph, a numbered section, a bylaw article — rather than embedded as a single vector per document. A whole-document embedding averages away the fine-grained distinctions a query needs to match against, so chunk-level granularity is what makes retrieval precise.

2. Embedding. An embedding model maps every chunk d_i and every incoming query q into a shared dense vector space,

$$\mathbf{q}, \mathbf{d}_i \in \mathbb{R}^n. \tag{1}$$

3. Retrieval. The system scores every chunk against the query and returns the top few by similarity.

4. Generation. The retrieved chunks, together with the original question, are passed to an LLM instructed to answer strictly from the supplied text and to cite the section it drew from.

Semantic retrieval: cosine similarity

The similarity measure used in step 3 is almost always cosine similarity between the query embedding and each chunk embedding:

$$\text{sim}(q, d_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \|\mathbf{d}_i\|} = \frac{\sum_{k=1}^n q_k d_{i,k}}{\sqrt{\sum_{k=1}^n q_k^2} \sqrt{\sum_{k=1}^n d_{i,k}^2}}. \quad (2)$$

The top- k chunks handed to the generator are then

$$\mathcal{D}_k(q) = \arg \max_{d_i \in \mathcal{D}}^{\text{top-}k} \text{sim}(q, d_i). \quad (3)$$

Lexical retrieval: BM25

Purely semantic search under-performs on legal and regulatory text specifically because embeddings are trained to capture paraphrase-level meaning, which can wash out the exact terms that matter in this domain — defined terms, article numbers, specific nouns that must appear verbatim. The standard remedy is to combine embedding search with a lexical score, typically BM25:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}, \quad (4)$$

$$\text{IDF}(t) = \ln \left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1 \right), \quad (5)$$

where $f(t, d)$ is the term frequency of term t in chunk d , $|d|$ is the chunk length, avgdl is the average chunk length across the corpus, N is the total number of chunks, and $n(t)$ is the number of chunks containing t . The parameters k_1 and b are the usual BM25 tuning constants controlling term-frequency saturation and length normalization, respectively.

Hybrid ranking

A final ranking blends the two normalized scores:

$$\text{score}(q, d_i) = \lambda \cdot \widehat{\text{sim}}(q, d_i) + (1 - \lambda) \cdot \widehat{\text{BM25}}(q, d_i), \quad \lambda \in [0, 1], \quad (6)$$

with $\widehat{\text{sim}}$ and $\widehat{\text{BM25}}$ each rescaled to $[0, 1]$ before combination. This hybrid search — dense retrieval for semantic matching, sparse retrieval for exact terms — is the single largest lever for retrieval accuracy on legal text, and it is worth implementing from the first version of the system rather than as a later optimization.

Why retrieval-then-generation suppresses hallucination

The generation step is the practical instantiation of the original RAG formulation of Lewis et al. (2020), in which the model marginalizes over the retrieved set:

$$p(y \mid x) \approx \sum_{z \in \mathcal{D}_k(x)} p(z \mid x) p(y \mid x, z). \quad (7)$$

In practice, no system explicitly computes this marginal — the top- k chunks are concatenated directly into the prompt context and the LLM conditions on all of them jointly. But the formulation is the theoretical justification for the architecture: conditioning generation on a small, retrieved, verifiable evidence set is what reduces hallucination relative to querying the LLM cold, because the model’s output distribution $p(y \mid x, z)$ is anchored to text that actually exists in the corpus rather than to whatever the model’s parameters happen to encode.

Practical stack (Python)

- **Embeddings** — OpenAI’s `text-embedding-3-*` family, Voyage AI (strong on legal and long-document retrieval specifically), or a local `sentence-transformers` model if document text should not leave the organization’s infrastructure.
- **Vector store** — Chroma or FAISS for a self-hosted, smaller-scale deployment; Pinecone, Weaviate, or pgvector (on Postgres) for a managed or SQL-integrated option.
- **Orchestration** — LangChain or LlamaIndex, which handle chunking, retrieval, and prompt assembly with minimal boilerplate.
- **Generation model** — a Claude- or GPT-4-class model, called with a system prompt that explicitly constrains it: answer only using the provided excerpts, state explicitly when the answer is not in them, and cite the section number.

Guardrails specific to legal and regulatory text

A few design choices matter enough, in this domain, to be non-optional from the first prototype rather than later hardening:

- Generation temperature should be kept near 0, so wording stays literal rather than creative.
- Every answer should surface the source clause or section alongside its plain-language explanation, so the reader can verify the claim rather than trust a paraphrase on faith.
- The system needs an explicit “I don’t know” path for low retrieval-confidence cases, rather than letting the model guess when nothing relevant was actually retrieved.
- A visible disclaimer — that the output is informational and not a substitute for legal advice — should ship with the system from day one; bylaws interpretation can carry real consequences, and that framing protects both the operator and the end user.